

Experiment No. 7

Write the Python program to implement DFS (Depth First Search)

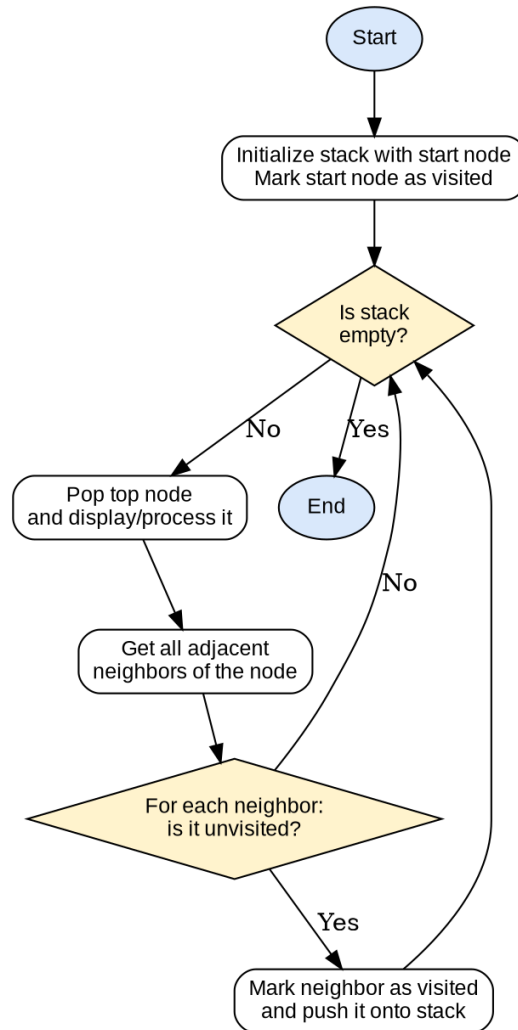
Aim

To write a Python program to implement the Depth First Search (DFS) algorithm for traversing a graph.

Algorithm

1. Start.
2. Represent the graph using an adjacency list.
3. Initialize an empty visited set and an empty traversal order list.
4. Call the recursive DFS function with the start node.
5. Mark the current node as visited and add it to the traversal order.
6. For every unvisited neighbor of the current node, recursively call DFS on that neighbor.
7. Repeat step 6 until all nodes reachable from the start node are visited.
8. Display the DFS traversal order.
9. Stop.

Flowchart



Program

```

def dfs(graph, start, visited=None, order=None):
    if visited is None:
        visited = set()
    if order is None:
        order = []

    visited.add(start)
    order.append(start)

    for neighbor in graph[start]:
        if neighbor not in visited:
            dfs(graph, neighbor, visited, order)
  
```

```
return order
```

```
graph = {  
    'A': ['B', 'C'],  
    'B': ['A', 'D', 'E'],  
    'C': ['A', 'F'],  
    'D': ['B'],  
    'E': ['B', 'F'],  
    'F': ['C', 'E']  
}
```

```
start_node = 'A'  
result = dfs(graph, start_node)  
print("DFS Traversal starting from node", start_node, ":")  
print(" -> ".join(result))
```

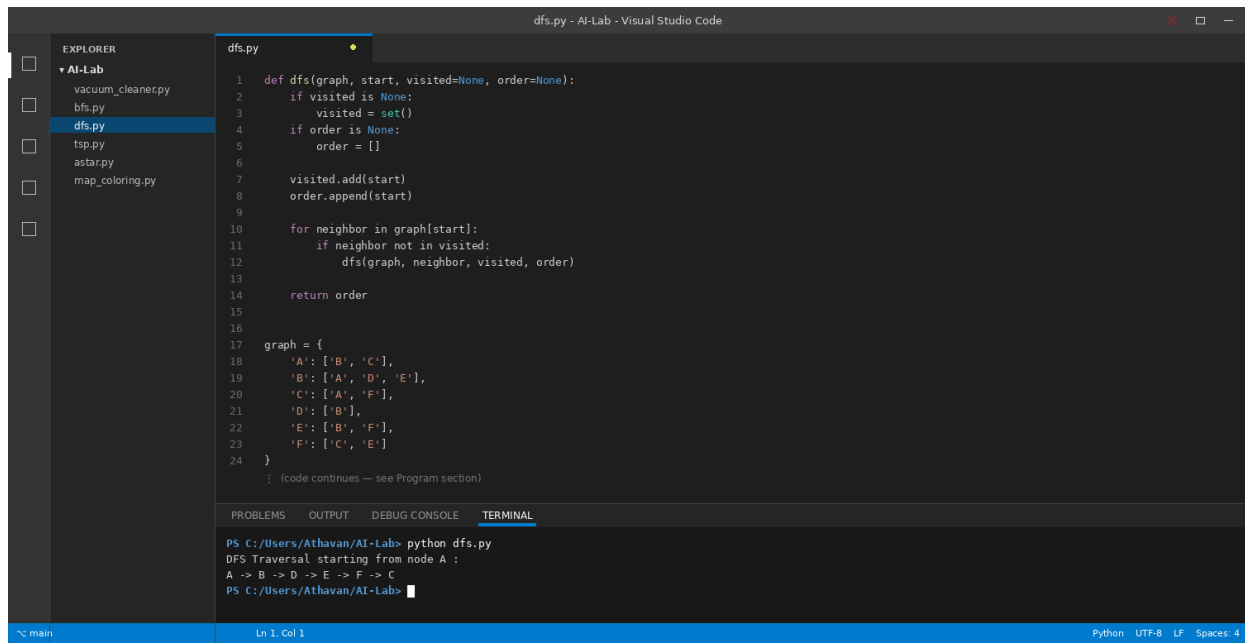
Result

The program was executed successfully and the output was verified as correct.

```
DFS Traversal starting from node A :  
A -> B -> D -> E -> F -> C
```

Sample Output — Running in VS Code

Screenshot of the program being run in the Visual Studio Code integrated terminal:



The image shows a Visual Studio Code window titled "dfs.py - AI-Lab - Visual Studio Code". The Explorer sidebar on the left shows a project named "AI-Lab" with several files: vacuum_cleaner.py, bfs.py, dfs.py (selected), tsp.py, astar.py, and map_coloring.py. The main editor displays the code for dfs.py, which implements a Depth-First Search (DFS) algorithm. The code defines a function dfs(graph, start, visited=None, order=None) that returns the order of nodes visited. A graph is defined with nodes A through F and their neighbors. The terminal at the bottom shows the command "python dfs.py" being executed, resulting in the output "DFS Traversal starting from node A : A -> B -> D -> E -> F -> C".

```
1 def dfs(graph, start, visited=None, order=None):
2     if visited is None:
3         visited = set()
4     if order is None:
5         order = []
6
7     visited.add(start)
8     order.append(start)
9
10    for neighbor in graph[start]:
11        if neighbor not in visited:
12            dfs(graph, neighbor, visited, order)
13
14    return order
15
16
17 graph = {
18     'A': ['B', 'C'],
19     'B': ['A', 'D', 'E'],
20     'C': ['A', 'F'],
21     'D': ['B'],
22     'E': ['B', 'F'],
23     'F': ['C', 'E']
24 }
25 : (code continues — see Program section)
```

PROBLEMS OUTPUT DEBUG CONSOLE **TERMINAL**

```
PS C:/Users/Athavan/AI-Lab> python dfs.py
DFS Traversal starting from node A :
A -> B -> D -> E -> F -> C
PS C:/Users/Athavan/AI-Lab>
```

Ln 1, Col 1 Python UTF-8 LF Spaces: 4